Big-data workflow checklist (quiz Section 3)

1. The 8-step checklist

1. **Size it.** Under ~10M rows *and* the file is under ~1/5 of RAM → pandas is OK. Bigger → **DuckDB / Polars** (or Dask). *(pandas needs ~5–10× the file size in RAM)*
2. **Peek first.** Look at a sample (`head`, a few raw rows) before loading everything.
3. **Read only what you need.** Only the needed columns (`usecols` / `SELECT`), right types. Save as **Parquet**.
4. **Filter early.** Drop unneeded rows (years, regions) *while reading*, not after.
5. **Clean dates & values.**
    - Negative or zero times, impossible dates (1900, 2099).
    - Missing codes (−9, 999, "$0.00"), duplicate rows.
6. **Shrink before you join.** Aggregate the big table first (sum, count), *then* merge small lookups.
7. **Merge safely.** See the checklist in section 3.
8. **Report honestly.**
    - Show the n behind each average, and use the median for skewed data.
    - Excel max = **1,048,576 rows**.
    - Paths that run on any computer.

> **One-liner: size it → peek → pick columns → filter → clean → shrink → join safely → report n.**

---

2. When does data get bigger? (rows vs memory)

Action	Rows	Memory
New column (`total_time = a - b`)	same	**+1 column** (50M values ≈ 0.4 GB each)
Lookup merge, **unique** keys	**same** (adds columns, not rows)	**+ a full new copy** of the table during the merge
Merge with **duplicate** keys on the lookup side	**more rows** (copies)	grows with the copies
Many-to-many merge (duplicates on both sides)	**explodes** (3 × 4 = 12)	explodes
`df = df.something()` (filter, sort, assign)	same or fewer	old + new exist briefly → **~2× peak**
Filter / aggregate	**fewer**	smaller

Key facts:

- A merge does **not** stack the 500K restaurant rows under the 50M. It **looks up** each delivery's restaurant and **adds its columns** to that row. So 50M in → 50M out, *if* each restaurant appears once.
- Rows multiply **only** when a key appears **more than once** in the lookup table.
- Even a perfect merge builds a **new** 50M-row table while the old one is still in memory, so it briefly takes about twice the memory. That's why you shrink first.

---

3. Merge checklist (before and after every join)

1. **Is the lookup key unique?** `restaurants.restaurant_id.is_unique` (or `.duplicated().sum() == 0`).

2. **Same key type on both sides?** int vs text, and **leading zeros** in zips/IDs (`"06510"` ≠ `6510`).
3. **Shapes before and after.** Print the row count. If it **grows** → duplicates. If it **shrinks** → unmatched rows dropped.
4. **Lock it.** `validate="many_to_one"` crashes instead of silently copying rows.
5. **Who didn't match?** `indicator=True`, then count and inspect `left_only`. Are the unmatched rows different (selection)?
6. **Merge only the columns you need** from the lookup (`restaurant_id, cuisine`), not the whole table.

---

4. "Suspicious result" checklist (the 8-minute star driver)

1. **How many observations?** A tiny n gives a noisy average.
2. **Bad values?** Zeros, negatives, impossible times; check the distribution and the share of values under 5 minutes.
3. **Duplicates?** The same order counted twice.
4. **Compare like with like.** Same restaurants, zips, hours (fixed-effects thinking).
5. **Mean vs median.** If they're far apart, outliers or bad data are driving the mean.
6. **Is it causal?** A link isn't a cause (selection, omitted variables).

---

5. Answer template for Section 3

- **A, problems:** name the problem → **the symptom** (kernel dies / `MemoryError` / freeze / rows jump / wrong averages).
- **B, efficiency:** name the change → **the principle** ("shrink before you join": select, filter, aggregate, then merge).
- **C, data quality:** name the suspect → **the check** you would run.